produces:

```
now *s The Time
```

Unlike sub and gsub, sub! and gsub! return the string only if the pattern was matched. If no match for the pattern is found in the string, they return nil instead. This means it can make sense (depending on your need) to use the ! forms in conditions.

Regular Expression Patterns

Like most things in Ruby, regular expressions are just objects—they are instances of the class Regexp. This means you can assign them to variables, pass them to methods, and so on:

```
str = "dog and cat"
pattern = /nd/
pattern.match?(str) # => true
str.match?(pattern) # => true
```

You can also create regular expression objects by calling the Regexp class's new method or by using the arbitrary delimiter %r{...} syntax. The %r syntax is particularly useful when creating patterns that contain forward slashes:

```
/mm\/dd/             # => /mm\/dd/
Regexp.new("mm/dd") # => /mm\/dd/
%r{mm/dd}            # => /mm\/dd/
```

Playing with Regular Expressions

If you're like us, you'll sometimes get confused by regular expressions. You create something that *should* work, but it just doesn't seem to match. That's when we fall back to irb. We'll cut and paste the regular expression into irb and then try to match it against strings. We'll slowly remove portions until we get it to match the target string and add stuff back until it fails. At that point, we'll know what we were doing wrong.

Another option is to use the website Rubular, at https://rubular.com. Rubular allows you to enter a regular expression and a test string and shows what the match result is. The site also allows you to create a permalink for your particular regular expression and test string which is excellent for using as a comment where the regular expression is defined in your code.

Regular Expression Options

A regular expression may include one or more options that modify the way the pattern matches strings. If you're using literals to create the Regexp object, then the options are one or more characters placed immediately after the terminator. If you're using Regexp.new, the options are constants used as the second parameter of the constructor.

i	*Case insensitive.*	The pattern match will ignore the case of letters in the pattern and string.
o	*Substitute once.*	Any #{...} substitutions in a particular regular expression literal will be performed just once, the first time it is evaluated. Otherwise, the substitutions will be performed every time the literal generates a Regexp object.